

A REPORT

ON

BANKING MANAGEMENT SYSTEMS

By

Name of the Student
K. Harini

Registration No.
AP24110010123

Prepared in the partial fulfilment of the
Project Based Learning of Course CSE 205 – Hands-On With Python



SRM UNIVERSITY, AP

APRIL 2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to everyone who supported us during the development of our **Banking Management System project**. We are thankful to SRM University, AP, for providing the resources and a supportive environment that helped us complete this work.

We are especially grateful to our **Faculty Mentor, Dr. Kartick Chandra Mondal** , for his guidance, helpful feedback, and constant encouragement throughout the project. His support made it easier for us to overcome challenges and improve our work

We are truly obliged to **Prof. Ch. Satish Kumar and Prof. C. V. Tomy**, Dean of the School of Engineering and Sciences (SEAS), for giving us the opportunity, support, and facilities needed to complete this project successfully.

We would also like to thank our team members for their cooperation and effort, which helped us handle the technical difficulties faced during the project. Lastly, we would like to thank our friends, peers, and family members for their continuous encouragement and support, which played an important role in finishing this work smoothly.

CERTIFICATION

This is to certify that the project entitled “Banking Management System” has been successfully completed by (K. Harini, AP24110010123) as a required academic component of the course CSE 205: Hands-On with Python during Semester 4 of Academic Year 2025-26 in the Department of Computer Science and Engineering at SRM University, AP. This work was executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of 07-Apr-26.

(Signature of the Course Instructor)

Dr. Kartick Chandra Mondal,
Asst Professor,
Department of Computer Science and Engineering
SRM University, AP

ABSTRACT

This Python program is a **console-based banking management system** designed to perform basic banking operations securely. It supports both user and admin functionalities through a login system with masked password input.

Users can perform **various banking operations** such as depositing and withdrawing money, transferring funds, viewing transaction history, calculating interest, and checking loan eligibility. Additionally, users have the option to delete their accounts when required. These features ensure smooth and practical handling of day-to-day banking activities within the system.

The program also includes an **admin panel** that enables the administrator to view and manage all user accounts, including the ability to delete accounts. Data is stored and retrieved using a text file (bank.txt), ensuring persistence across sessions. Overall, the project demonstrates the effective use of file handling, dictionaries, user authentication, and menu-driven programming in Python.

CONTENTS	PAGE. NO.
1.1. COVER PAGE	1
1.2. ACKNOWLEDGEMENTS	2
1.3. CERTIFICATION	3
1.4. ABSTRACT	4
1.5. TABLE OF CONTENTS	5
1.6. INTRODUCTION	6
1.7. METHODOLOGY	7-8
1.8. ALGORITHM	9-11
1.9. CODE	12-15
1.10. FINAL OUTPUT	16-20
1.11. OUTPUT EXPLANATION	21
1.12. CONCLUSION/RECCOMENDATIONS	22
1.13. REFERENCES	23

INTRODUCTION

Banking systems play an essential role in managing financial transactions efficiently and securely. With the advancement of technology, digital banking solutions have become increasingly important for handling user accounts and transactions in a systematic way.

This project focuses on developing a basic banking management system using Python to simulate real-world banking operations. It is designed to perform basic banking operations like creating an account, depositing money, withdrawing money, and transferring funds between users. The system works through a menu-driven program, making it easy for users to interact with.

The program stores all account details such as username, password, balance, and transaction history in a file, so the data is not lost even after closing the program. It also includes a login system for both users and an admin. The admin has special permissions to view and delete user accounts.

One of the important features of this system is password security, where the password is hidden while typing. By implementing features such as user authentication, transaction handling, and data storage using file handling, this project provides a practical understanding of how banking systems function.

METHODOLOGY

The banking management system is developed using fundamental Python concepts such as **functions, loops, conditional statements, dictionaries, and file handling**. The program begins by loading existing account data from a file (`bank.txt`) using the `load_data()` function. If data is available, it is converted into a dictionary and stored in memory; otherwise, an empty structure is initialized. The system follows a menu-driven approach, allowing users to continuously interact with options like account creation, login, and exit through a loop-based structure.

When a user **creates an account**, the system collects details such as username, password, and initial balance. The password is entered securely using masking techniques. The data is stored in a dictionary and saved to the file using the `save_data()` function.

The **login process** verifies the user's credentials by comparing the entered username and password with the stored data. If the credentials match, access is granted. There are two types of login:

- **Admin login:** It uses predefined credentials and provides access to the admin panel
- **User login:** It allows access to personal banking operations

After successful login, the system redirects the user to the appropriate menu. In the **user menu**, several operations are available:

- **Balance:** Displays the current account balance of the user
- **Deposit:** Adds a specified amount to the user's balance and records the transaction
- **Withdrawal:** Deducts an amount if sufficient balance is available
- **Fund Transfer:** Transfers money from one user to another after validating the receiver and balance
- **Transaction History:** Displays all previous transactions stored in the list
- **Interest Calculation:** Calculates simple interest based on a fixed rate and time
- **Loan Eligibility Check:** Determines whether the user is eligible for a loan based on their balance

- **Delete Account:** Allows users to permanently remove their account after confirmation

Each operation updates the user's data and immediately saves the changes to the file to ensure data consistency and persistence.

The **admin panel** provides higher-level control over the system. The admin can view all user accounts in the system and delete any account if required. This helps in managing and maintaining the database effectively.

The program ensures that all data is properly saved before exiting to prevent data loss. Overall, the methodology focuses on modular programming using functions, efficient data handling with dictionaries, and a structured menu-driven interface to ensure smooth and user-friendly operation.

ALGORITHM

BEGIN

1. Initialize empty dictionary accounts
 2. Call load_data()
 - 2.1. IF bank.txt exists
 - 2.1.1. Read file data
 - 2.1.2. Convert into dictionary
 - 2.2. ELSE
 - 2.2.1. Initialize empty dictionary
-

3. REPEAT

4. Display main menu:
 - 4.1. Create Account
 - 4.2. Login
 - 4.3. Exit
 5. Input user choice
-

6. IF choice = 1 (Create Account)

7. Input username
 8. Input password (masked input)
 9. Input initial balance
 10. Store details in dictionary
 11. Save data to file
-

12. ELSE IF choice = 2 (Login)

11. Input username and password
12. IF admin credentials
 - 12.1. Go to Admin Panel

13. ELSE IF valid user

13.1. Go to User Menu

14. ELSE

14.1. Display "Invalid Login"

User Menu

15. Display options:

15.1. View Balance

15.2. Deposit

15.3. Withdraw

15.4. Transfer

15.5. Transaction History

15.6. Interest Calculation

15.7. Loan Check

15.8. Delete Account

15.9. Logout

16. Input user choice

17. Perform operation:

17.1. **View Balance** - Display current balance

17.2. **Deposit** - Add amount and save data

17.3. **Withdraw** - Check balance, deduct, save data

17.4. **Transfer** - Verify user & balance, update both accounts

17.5. **Transaction History** - Display all records

17.6. **Interest** - Calculate and display interest

17.7. **Loan Check** - Check eligibility

17.8. **Delete Account** - Confirm and remove account

17.9. **Logout** - Exit user m

Admin Panel

18.Display options:

18.1. View all accounts

18.2. Delete account

18.3. Logout

19.Input admin choice

20.Perform operation:

20.1. View all users

20.2. Delete selected account and save data

20.3. Logout

ELSE IF choice = 3 (Exit)

21.Save data to file

22.Terminate program

23.**UNTIL** user selects Exit

END

CODE

```
code.py - E:\SEM 2-2\PY\project\code.py (3.10.9)
File Edit Format Run Options Window Help

accounts = {}
import msvcrt
import sys

def input_password(prompt):
    # If running in IDLE - use normal input
    if "idlelib" in sys.modules:
        return input(prompt)

    # Otherwise use masked input (CMD)
    import msvcrt
    print(prompt, end="", flush=True)

    password = ""

    while True:
        ch = msvcrt.getch()

        if ch == b'\r':
            print()
            break
        elif ch == b'\x08':
            if len(password) > 0:
                password = password[:-1]
                print("\b \b", end="", flush=True)
        elif ch in (b'\x00', b'\xe0'):
            msvcrt.getch()
        else:
            password += ch.decode("utf-8")
            print("*", end="", flush=True)

    return password

# load data
def load_data():
    global accounts
    try:
        file = open("bank.txt", "r")
        data = file.read()
        if data != "":
            accounts = eval(data)
        file.close()
    except:
        accounts = {}

# save data
def save_data():
    file = open("bank.txt", "w")
    file.write(str(accounts))
    file.close()

# login system
def login():
    username = input("Enter username: ")
    password = input_password("Enter password: ")

    if username == "admin" and password == "admin":
        print("Admin Login Successful")
        return "admin"
    elif username in accounts and accounts[username]["password"] == password:
        print("User Login Successful")
        return username
    else:
        print("Invalid login")
        return None
```

```

# withdraw
def withdraw(user):
    amount = int(input("Enter amount: "))
    if accounts[user]["balance"] >= amount:
        accounts[user]["balance"] -= amount
        accounts[user]["transactions"].append("Withdrew " + str(amount))
        save_data()
        print("Amount withdrawn!")
    else:
        print("Insufficient balance!")

# fund transfer
def transfer(user):
    to_user = input("Enter receiver username: ")
    amount = int(input("Enter amount: "))

    if to_user in accounts:
        if accounts[user]["balance"] >= amount:
            accounts[user]["balance"] -= amount
            accounts[to_user]["balance"] += amount

            accounts[user]["transactions"].append("Transferred " + str(amount) + " to " + to_user)
            accounts[to_user]["transactions"].append("Received " + str(amount) + " from " + user)

            save_data()
            print("Transfer successful!")
        else:
            print("Insufficient balance!")
    else:
        print("Receiver not found!")

# transaction history
def show_transactions(user):
    print("Transaction History:")
    for t in accounts[user]["transactions"]:
        print(t)

# interest
def calculate_interest(user):
    rate = 5
    time = 1
    balance = accounts[user]["balance"]
    interest = (balance * rate * time) / 100
    print("Interest:", interest)

# loan check
def loan_check(user):
    if accounts[user]["balance"] > 5000:
        print("Eligible for loan")
    else:
        print("Not eligible")

# delete account
def delete_account(user):
    confirm = input("Are you sure? (yes/no): ")
    if confirm == "yes":
        del accounts[user]
        save_data()
        print("Account deleted!")
        return True
    return False

```

```
# main menu
load_data()

while True:
    print("\n1. Create Account")
    print("2. Login")
    print("3. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        create_account()

    elif choice == 2:
        user = login()
        if user == "admin":
            admin_panel()
        elif user:
            user_menu(user)

    elif choice == 3:
        save_data()
        break

# user menu
def user_menu(user):
    while True:
        print("\n- USER MENU -")
        print("1. View Balance")
        print("2. Deposit")
        print("3. Withdraw")
        print("4. Transfer")
        print("5. Transaction History")
        print("6. Interest")
        print("7. Loan Check")
        print("8. Delete Account")
        print("9. Logout")

        ch = int(input("Enter choice: "))

        if ch == 1:
            view_balance(user)

        elif ch == 2:
            deposit(user)

        elif ch == 3:
            withdraw(user)

        elif ch == 4:
            transfer(user)

        elif ch == 5:
            show_transactions(user)

        elif ch == 6:
            calculate_interest(user)

        elif ch == 7:
            loan_check(user)

        elif ch == 8:
            if delete_account(user):
                break

        elif ch == 9:
            break
```

```
# admin panel
def admin_panel():
    while True:
        print("\n- ADMIN PANEL -")
        print("1. View All Accounts")
        print("2. Delete Any Account")
        print("3. Logout")

        ch = int(input("Enter choice: "))

        if ch == 1:
            print(accounts)

        elif ch == 2:
            user = input("Enter username to delete: ")
            if user in accounts:
                del accounts[user]
                save_data()
                print("Account deleted by admin")
            else:
                print("User not found")

        elif ch == 3:
            break
```

NOTE: Password masking is implemented using msvcrt in command prompt. In IDLE, normal input is used due to environment limitations, ensuring compatibility across platforms.

FINAL OUTPUT

1. Create Account:

```
1. Create Account
2. Login
3. Exit
Enter choice: 1
Create username: s079
Create password: *****
Enter initial balance: 200
Account created successfully!
```

2. Login into your Account:

```
1. Create Account
2. Login
3. Exit
Enter choice: 2
Enter username: s079
Enter password: *****
User Login Successful
```

3. Unsuccessful Login:

```
1. Create Account
2. Login
3. Exit
Enter choice: 2
Enter username: s079
Enter password: *****
Invalid login
```

4. Successful Login - Display User Menu:

```
- USER MENU -
1. View Balance
2. Deposit
3. Withdraw
4. Transfer
5. Transaction History
6. Interest
7. Loan Check
8. Delete Account
9. Logout
Enter choice: 1
Current Balance: 200
```


5. Deposit Operation:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 2  
Enter amount: 300  
Amount deposited!
```

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 1  
Current Balance: 500
```

6. Withdraw Operation:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 3  
Enter amount: 100  
Amount withdrawn!
```

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 1  
Current Balance: 400
```

7. Transfer Operation - Fail:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 4  
Enter receiver username: s215  
Enter amount: 50  
Receiver not found!
```

8. Transfer Operation - Success:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 4  
Enter receiver username: s129  
Enter amount: 200  
Transfer successful!
```

9. Transaction History Operation:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 5  
Transaction History:  
Deposited 300  
Withdrew 100  
Transferred 200 to s129
```

10. Interest Operation:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 6  
Interest: 10.0
```

11. Loan Check Operation:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 7  
Not eligible
```

11. Logout Operation:

```
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 9  
  
1. Create Account  
2. Login  
3. Exit  
Enter choice: |
```

Saved Operations / Data:

```
{'s123': {'password': '24110010123', 'balance': 320, 'transactions': ['Withdrew 200', 'Deposited 20']},  
{'s129': {'password': '24110010129', 'balance': 700, 'transactions': ['Received 200 from s079']},  
{'s079': {'password': '24110010079', 'balance': 200, 'transactions': ['Deposited 300', 'Withdrew 100', 'Transferred 200 to s129']}}
```

12. Delete Account Operation:

```
1. Create Account  
2. Login  
3. Exit  
Enter choice: 2  
Enter username: s129  
Enter password: *****  
User Login Successful  
  
- USER MENU -  
1. View Balance  
2. Deposit  
3. Withdraw  
4. Transfer  
5. Transaction History  
6. Interest  
7. Loan Check  
8. Delete Account  
9. Logout  
Enter choice: 8  
Are you sure? (yes/no): yes  
Account deleted!
```

Saved Operations / Data after performing Delete Operation:

```
{'s123': {'password': '24110010123', 'balance': 320, 'transactions': ['Withdrew 200', 'Deposited 20']},  
{'s079': {'password': '24110010079', 'balance': 200, 'transactions': ['Deposited 300', 'Withdrew 100', 'Transferred 200 to s129']}}
```

OUTPUT EXPLANATION

1. The program allows users to create new bank accounts.
2. It provides a secure login system
 - Separate login for admin and users
 - Password is hidden while typing
3. Users can deposit money.
4. Users can withdraw money
 - Checks if sufficient balance is available
 - Updates balance after withdrawal
5. The system supports fund transfer between users.
6. Transaction history is maintained
 - Displays all past transactions
 - Helps in tracking account activity
7. The program calculates simple interest.
8. Loan eligibility is checked based on account balance.
9. Users can delete their accounts
 - Confirmation is required before deletion
10. Admin functionalities are provided
 - View all user accounts
 - Delete any user account
11. All data is stored in a file and retained after execution.
12. The system uses a menu-driven interface for easy interaction.

CONCLUSION

The Banking Management System developed using Python successfully demonstrates the implementation of basic banking operations in a simple and efficient manner. The system allows users to create accounts, perform transactions, and manage their financial activities while ensuring data persistence through file handling.

The project also includes essential features such as user authentication, transaction tracking, and an admin panel for managing accounts. By using concepts like dictionaries, functions, and a menu-driven approach, the system provides a clear understanding of how real-world banking applications work.

Overall, this project serves as a good learning experience for understanding Python programming, data management, and system design, while also providing a foundation for developing more advanced and secure banking applications in the future.

RECOMMENDATIONS

1. Implement **GUI (Tkinter)** for better user experience
2. Use **MySQL** instead of text file for efficient data management
3. Apply **password encryption** for improved security
4. Add **features** like balance enquiry, mini statement, account update
5. Implement input validation to **avoid errors**
6. Improve **error handling** for system stability
7. Add **OTP/two-factor authentication** for better security
8. Extend system to **web or mobile application**

REFERENCES:

- [Python Tutorial – W3Schools](#)
- [Python File Handling – W3Schools](#)
- [Python File Write/Create – W3Schools](#)
- [Python open\(\) Function – W3Schools](#)
- [Python File Methods – W3Schools](#)

THANK YOU